TESTNET EDITION

Klaro.

A USDC transfer is fast, final, and useless to a real business. Klaro turns it into an **invoice, a payment, and an on-chain receipt anyone can verify** — built on Circle's Arc.

● VERIFIED RECEIPT · 0x19d4…2B00 · Arc testnet

DOCUMENT
Klaro product paper
Investor & partner brief

NETWORK
Arc testnet · chain 5042002
klaro-peach.vercel.app

DATE
June 2026
Klaro Labs

01 The problem: a stablecoin transfer is not a payment

Two wallets, one USDC transfer. It clears in under a second and it is irreversible. For a real business, that is where the trouble starts, not where it ends.

A bank wire or a card capture carries more than money. It carries a receipt the recipient can show an accountant, an audit trail a regulator can follow, a screening check that keeps sanctioned counterparties out, and a path back to the local currency the recipient actually spends. A raw on-chain transfer carries none of that. The money lands; everything around the money is missing.

What the transfer leaves behind

- **No verifiable receipt.** A transaction hash proves a transfer happened. It does not prove *what it was for* — which invoice, which terms, which counterparty.
- **No audit trail.** At year end there is a block explorer, not a ledger an accountant or a lender can reconcile.
- **No screening.** Every regulated payment provider screens counterparties before settlement. A wallet-to-wallet send skips that step entirely.
- **Stranded value.** The vendor holds USDC. The grocer, the landlord, and the tax authority want local currency.

Who feels it

The vendor billing across borders — a developer in Pune, an agency in São Paulo, a freelancer in Lagos — invoices a client overseas and is told to "just take USDC." They do. Then they cannot prove the payment cleanly to their bank, cannot file it without a paper trail, and cannot spend it without an off-ramp that screens the source.

The instrument is fast and final. As a payment for a business, it is incomplete.

> Klaro's wedge is the missing layer: the same fast, final transfer, wrapped in an invoice, a settlement state machine, and a receipt that carries proof off-chain to a bank or a tax filing.

02 The product: invoice to verifiable receipt

Klaro is Arc-native USDC invoicing with on-chain receipts, vendor reputation, and a partner cashout path. One flow, end to end, with every leg labelled for exactly what it is.

01	02	03	04	05
Invoice	**Pay**	**Screen & settle**	**Receipt**	**Cash out**
Vendor signs in with Google or a magic link and issues an invoice. Hosted page, fixed terms.	Buyer pays from any wallet; cross-chain pay-in routes through CCTP V2 and Circle Gateway.	Counterparty is screened, then the operator settles escrow. No screen, no release.	An audit-grade receipt is minted on chain and resolves at a public verify URL.	Vendor previews local-currency cashout through an LP network — fiat leg simulated on testnet.

The five surfaces

Each surface is a distinct product seam with its own users, contracts, and honest-mode label. The web app exposes them as separate portals; the SDK, CLI, and embeds expose them to integrators.

S1
Invoicing & pay-in
Vendor invoices, hosted pay pages, wallet payment, cross-chain routing.

S2
Receipts
On-chain audit receipt, public verify URL, embeddable badge.

S3
Reputation
Vendor reputation from settlement and dispute history, mirrored on chain.

S4
LP & cashout
Liquidity-provider staking, proof, dispute, slashing; cashout simulation.

S5
Agents
ERC-8004 identity, ERC-8183 job escrow, budget-capped agent wallets.

> The receipt is the moat. A fiat processor cannot issue an artifact a third party can verify without trusting the processor. A Klaro receipt resolves against on-chain state from its hash alone — the verifier trusts the chain, not Klaro.

03 Why Arc

A payment product wants a settlement layer that behaves like settlement: one unit of account, deterministic finality, and native compliance and treasury rails. Arc is built that way.

USDC is the gas token

On Arc, gas is paid in USDC — the same asset the invoice is denominated in. There is no second volatile token to acquire, hold, or explain to a vendor. The full 20-contract Klaro protocol deployed for **0.55 USDC** of gas.

Accounting note: app balances use the ERC-20 USDC representation at 6 decimals; the native gas representation is kept strictly separate in code. The two are never mixed.

Sub-second deterministic finality

Settlement is final in under a second with no probabilistic confirmation window. The product is designed around that — a paid invoice is paid, not "paid pending twelve blocks."

Circle-native by construction

Klaro builds on Circle's own primitives rather than re-implementing them:

PRIMITIVE	USE IN KLARO
CCTP V2	`cross-chain pay-in`
Circle Gateway	`batched settlement`
Circle Wallets	`passkey vendor wallets`
StableFX	`USDC ↔ EURC corridor`
Permit2	`gasless allowances`
ERC-8004 / 8183	`agent identity + escrow`

Cross-chain transfers use CCTP V2 only. External Arc and Circle addresses are pinned in `KlaroConfig.sol` and drift-checked in CI against the live Arc docs corpus.

> Arc is a testnet today; mainnet is not assumed anywhere in the codebase. Every Arc-touching path either runs live on testnet or falls back to a labelled simulator. Nothing in this document depends on a mainnet that does not yet exist.

04 Architecture

Three runtimes, one contract layer, one database. The web app never signs money-moving transactions; the daemon holds the only operator key.

20
CONTRACTS DEPLOYED

15
DAEMON WORKERS

42
TABLES, RLS ON EACH

Web — Next.js 15 App Router

Marketing at the apex domain; a vendor portal under `/vendor/*`, an LP portal under `/lp/*`, and an admin console under `/admin/*`. Public money pages — `i/[id]`, `pay/[slug]`, `receipt/[hash]` — are the only routes a buyer or verifier touches without an account. Sessions are Supabase SSR (Google OAuth and magic link); CSP, subdomain rewrites, and rate limiting live in middleware.

Daemon — the operator process

A long-lived Node process pairs an Arc event listener (`arcSubscriber.ts`) with fifteen BullMQ worker classes — screen-and-settle, cashout advance, receipt generation, dispute resolution and decision, reconciliation, reputation, FX, ERP sync, notifications, and more — each with retries and a dead-letter queue watcher over every queue it owns. The daemon holds `KLARO_OPERATOR_PRIVATE_KEY`; the web app holds no signing key. Listener idempotency keys are `(event, txHash, logIndex)`, so a replayed log never double-settles.

Contracts — 20 on Arc testnet

Each contract is scoped to one concern and tested in isolation. A local forge run on 2026-06-10 verified 531 passing Foundry tests across unit, fuzz, and a deploy-wiring regression suite that re-runs the full deploy and asserts every permission. Echidna and Halmos harnesses are scaffolded but not yet wired, so they are not counted as coverage.

MONEY FLOW	CASHOUT & LP	AGENTS	DISPUTES & REP	CROSS-CHAIN / FX	PRIVACY / CONFIG
`InvoiceEscrow`	`CashoutOrderProcessor`	`AgentRegistry`	`DisputeManager`	`MultiChainRouter`	`CounterpartyRegistry`
`AuditReceipt`	`LPStaking`	`AgentEscrow`	`ReputationManager`	`StableFXAdapterRegistry`	`PrivacyVeil`
`RefundProtocol`	`LPRegistry`	`AgentBudgetWallet`	`VendorReputation`		`KlaroConfig`
`FeeSplitter`	`ProofRegistry`				
`RoutePolicyEngine`	`RetainerStream`				

Data & integrators

Supabase Postgres with row-level security on every table, enforced through `current_vendor_id()` and append-only SQL migrations. The `@klaro/sdk` TypeScript client, a CLI, an embeddable receipt badge, and an iframe-friendly hosted invoice page expose the same flows to integrators.

Test count (531) verified by a local forge run on 2026-06-10; worker count read from daemon source. The daemon's boot log carries a stale literal worker count; the authoritative figure is the 15 worker classes started at boot.

05 Trust design

A payment product earns trust by being precise about what it does and what it does not. Klaro encodes that precision in the UI, the contracts, and the data model — not in marketing copy.

Honest-mode labelling

Every surface tells the user what mode it is in. Klaro never ships UI that pretends to be more than it is, and never ships a control that does nothing.

LABEL	MEANING
`live testnet`	End-to-end on Arc testnet
`simulated`	Mock store, no chain calls
`access pending`	Adapter built; credentials not yet issued
`partner pending`	Integration coded; partner signature outstanding
`mainnet only`	Path exists on mainnet; testnet uses mock

Screening-gated settlement

Escrow does not release until the counterparty is screened. On testnet the screen runs in simulation; it becomes a live check only when a provider key (Chainalysis, TRM, or Sumsub) is added and verified. The gate exists in code today; the verdict is simulated until then.

Escrowed, traceable value

Every dollar of live testnet value sits in escrow and is traceable end to end. Funds move through `InvoiceEscrow`, release on a screened settlement, and split via `FeeSplitter` — no off-ledger hops.

Dispute & slashing state machine

Disputes run opener → evidence → operator review → on-chain decision, mirrored into `VendorReputation`. LP misbehavior is handled by an explicit staking, proof, and slashing path. Money flows are state machines with defined failures, retries, and final outcomes — never claim-based releases.

No PII on chain · append-only audit

The chain stores hashes, addresses, IDs, and status events only — never personal data. Off-chain, audit logs are append-only, and the logger strips emails and wallet addresses before any breadcrumb leaves the process.

> **Klaro is not a bank.** Financing readiness is not a loan offer. External smart-contract audits are not yet complete and are a hard gate before mainnet. The fiat cashout leg is simulated on testnet.

System threat model and controls — cross-tenant isolation, webhook replay and forgery, operator-secret containment, RPC compromise, PII-to-telemetry leakage — are documented in `THREAT_MODEL.md` and the contract-level threat model.

06 What is real today

The single most important table in this document. Read it before any claim elsewhere.

CAPABILITY	STATUS	WHAT THAT MEANS
Vendor signup & invoice creation	`live testnet`	Google OAuth or magic link, server-side identity, RLS-isolated per tenant.
Buyer payment	`live testnet`	Hosted invoice pages and wallet connection. Live or simulated path depends on the deployed contract env.
On-chain audit receipt	`live testnet`	Shareable verify URL; on-chain minting runs when the live receipt contract is configured.
LP staking, proof, dispute, slashing	`live testnet`	Full state machine is real testnet state.
Cross-chain pay-in	`live testnet`	CCTP V2 and Circle Gateway routing via `MultiChainRouter`.
StableFX corridor (USDC↔EURC)	`live testnet`	Routed through Circle's FxEscrow and Permit2; the adapter on testnet is a mock seeded into the registry.
Agent jobs (ERC-8004 / 8183)	`live testnet`	Budget-capped agent wallets, dispute-protected job escrow.
Fiat cashout (local currency payout)	`simulated`	The off-ramp leg is a testnet partner simulation. No real INR or other fiat moves.
Sanctions screening	`simulated`	Gate runs in simulation until a provider key (Chainalysis / TRM / Sumsub) is added and verified.
KYB onboarding (Sumsub-gated)	`access pending`	Adapter built; provider credentials not yet issued.
Card on-ramp (MoonPay), Wallet passes	`partner pending`	Integration coded; partner signature outstanding.
Real cashout corridors, multisig handover, bug bounty	`mainnet only`	Gated behind the external audit and mainnet launch.
External smart-contract audit	`not done`	Required before mainnet. Echidna/Halmos harnesses scaffolded, not yet wired.

> Read it plainly: the protocol, escrow, receipts, LP state, agents, and cross-chain pay-in are live on testnet. The fiat payout and live sanctions verdict are simulated. Real-money corridors and an audit sign-off come before mainnet, not before this document.

07 Go-to-market

Start where the pain is sharpest and the alternatives are worst: emerging-market vendors and freelancers invoicing clients overseas.

The wedge user

A vendor in an emerging market bills a client in the US, EU, or UK and today loses days and a meaningful slice of revenue to legacy transfer rails — or fails the structural requirements of a US-incorporation on-ramp altogether. They can receive USDC; what they lack is the receipt, the screening, and the clean path to local currency that turns USDC into a usable payment.

Klaro meets them at the invoice. The vendor never has to explain blockchains to their client: the client opens a hosted page and pays from a wallet, the vendor gets a verifiable receipt, and the off-ramp preview shows what local currency the settlement would convert to.

The LP side

Local-currency liquidity for cashout comes from a network of liquidity providers. That network is **invite-only**: LPs stake, post proof, and are subject to dispute and slashing. The design treats LP onboarding as a trust-and-risk decision, not a self-serve signup.

On testnet, LP staking and the dispute/slashing machine are real state; the fiat payout the LP would make is simulated. The mechanism is built and exercisable now; the cash leg goes live with real corridors at mainnet.

Distribution surface

The SDK, CLI, receipt badge, and invoice embed let other products issue Klaro invoices and verify Klaro receipts — the receipt verify URL is a public artifact, which makes every settled invoice a small distribution event.

Market sizing, per-country regulatory posture, and pricing mechanics are framed in the internal PRD. Testnet reality governs this document: figures that depend on real volume or live corridors are out of scope until those legs are live.

08 Roadmap

Three stages, gated by trust rather than dates. Each stage ships only when the prior one is real.

STAGE	WHAT SHIPS
Now `testnet`	Vendor invoicing, buyer payment, on-chain receipts, LP staking, partner cashout simulation, agent jobs, disputes, and cross-chain pay-in — all live on Arc testnet.
Next `hardening`	External contract audit; Sumsub-gated KYB; Chainalysis live sanctions screening; MoonPay card on-ramp; Apple and Google Wallet passes. This is where the simulated and pending legs become live.
Mainnet `gated`	Multisig ownership handover for every contract; Immunefi bug bounty live; real cashout corridors with off-ramp partners. None of this ships before the audit lands.

The mainnet gates, explicitly

- External smart-contract audit complete and findings resolved.
- Ownership of every `Ownable` contract transferred from the deployer EOA to a Safe multisig in the deploy broadcast — including the FX adapter, so a post-handover key compromise cannot drain destination liquidity.
- Live sanctions provider wired and verified, replacing the simulated verdict.
- Real off-ramp corridors signed, replacing the cashout simulation.

What this document does not promise

No dates. No mainnet behavior described as if it exists. No real-money movement, payout, lending, or completed audit. The roadmap is a sequence of gates; the only thing live today is the testnet column.

● AUDIT GATE · required before mainnet

09 Team, contact & deployment

Team

Prateek Tripathi
Klaro Labs

CONTACT
prateek@myklaro.app
github.com/klaro-labs/klaro
myklaro.app

Try the testnet

Live testnet: klaro-peach.vercel.app. The app boots without environment variables — every external surface falls back to a labelled simulated mode, so the full UI is navigable on a fresh clone. Wire any single surface live by filling in only the env it needs.

Deployment — Arc testnet

Chain 5042002 · RPC rpc.testnet.arc.network · release v0.1.0 · deploy gas 0.55 USDC. On testnet the deployer EOA retains ownership; mainnet hands every contract to a multisig in the same broadcast.

CONTRACT	ADDRESS
InvoiceEscrow	0xA76edAd6e1c0D0854a21BF527086CA44b620c4e2
AuditReceipt	0x19d44E987DBd853c3C94A4Ab35404cCCd7612B00
RefundProtocol	0xCC4cFb95ae8d5774DF66a70E2Af8aaD7A5076339
FeeSplitter	0x3b2E07e58f1578cF24B6438E3E76728C21555B66
CashoutOrderProcessor	0x347935A89B95fD2baD736dbADe4C14b0a5e9E6bd
LPStaking	0x4b36eD428b47F4254737215454BE6e9b99A1bD1f
ProofRegistry	0xb0a2c7815D75EeBF73f8869C810EC8da5FcCbC33
DisputeManager	0xee9561BE93312625C7F622D3f63B9092Af23aE5F
MultiChainRouter	0xAF636EbC33D9FCB124E21C567F174f1EA5e2A241
VendorReputation	0xb44CE869978CC1C0bf71687B307b19657d907750
PrivacyVeil	0x73660E5aa28a304369B1C9aF06d18468Af6a95F5

Full 20-contract address list and the deploy command that produced them are in DEPLOYMENT.md.

One-line summary. Klaro makes a USDC transfer behave like a payment a business can use — invoice, screened settlement, on-chain receipt, and an off-ramp preview — live on Arc testnet, honest about every leg, audit-gated before mainnet.